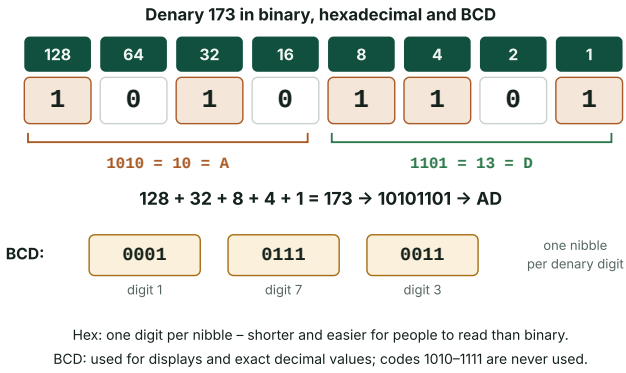
Number systems: bits, bytes, binary, denary, hexadecimal and BCD

FD01 FACT FILE

Specification: students should be able to

- Explain the terms bit, nibble, byte, kilobyte, megabyte, gigabyte and terabyte
- Demonstrate that 2^n different values can be represented with n bits
- Convert unsigned integers between denary and binary (up to 8 bits), showing working
- Convert between binary, denary and hexadecimal and explain why hexadecimal is used
- Describe binary-coded decimal (BCD) and its uses

Key words: bit, nibble, byte, kilobyte (KB), megabyte (MB), gigabyte (GB), terabyte (TB), place value, base 2 (binary), base 10 (denary), base 16 (hexadecimal), most significant bit (MSB), least significant bit (LSB), leading zeros, binary-coded decimal (BCD), colour code



173 converted to binary using place values, split into nibbles to give hexadecimal AD, and stored digit by digit in binary-coded decimal

Bits, bytes and number bases

Everything a computer stores – numbers, text, pictures, sound and the program instructions themselves – is held as patterns of bits, because the transistors and storage cells inside the machine have two reliable states. A single bit holds a 0 or a 1; four bits make a nibble and eight bits make a byte. A number base tells you how many different digits are available and what each column is worth. Denary (base 10) uses the digits 0 to 9 with column values that are powers of 10. Binary (base 2) uses only 0 and 1, with column values that double each time: 1, 2, 4, 8 and so on up to 128 in a byte. Hexadecimal (base 16) uses 0 to 9 followed by A to F for the values 10 to 15, with column values 1, 16, 256 and so on. The leftmost bit of a binary number has the highest place value and is called the most significant bit (MSB); the rightmost is the least significant bit (LSB). The same bit pattern can mean different things depending on the representation being used, which is why the rest of this topic is about agreed ways of interpreting bits.

Units of storage

Memory, disk capacity and file sizes are measured in bytes, and larger units are formed by multiplying by 1024, which is 2^{10}. A kilobyte is 1024 bytes, a megabyte is 1024 kilobytes (2^{20} bytes), a gigabyte is 1024 megabytes (2^{30} bytes) and a terabyte is 1024 gigabytes (2^{40} bytes). Each unit is therefore roughly a thousand times the one below, so a megabyte is about a million bytes and a gigabyte is about a billion. To convert to a larger unit divide by 1024; to convert to a smaller one multiply by 1024; to change bits into bytes divide by 8. Be aware that hard drive manufacturers and the international standards body use powers of 1000 for KB, MB and GB, and use the names kibibyte (KiB), mebibyte (MiB) and so on for the powers of 1024. This is why a new '1 TB' drive appears smaller when the operating system reports it. For CCEA answers, use the 1024-based definitions unless a question tells you otherwise, and always state the unit with your final answer.

Counting with n bits

Each extra bit doubles the number of different patterns available, because every existing pattern can be followed by either a 0 or a 1. One bit gives 2 patterns, two bits give 4, three bits give 8, and in general n bits give 2^n different values. As unsigned integers these run from 0 to $2^n - 1$, so a byte stores 0 to 255 and a 10-bit field stores 0 to 1023. The reverse question – how many bits are needed? – is answered by finding the smallest power of 2 that is at least the number of values required. To store every whole number from 0 to 1000 you need 1001 different values; 2^9 is only 512, but 2^{10} is 1024, so 10 bits are needed. Watch the off-by-one trap: storing 0 to 16 777 215 needs 16 777 216 values, which is exactly 2^{24}, so 24 bits are enough. Leading zeros are normally kept when numbers are stored or transmitted in a fixed width, because the receiving system must know exactly where one byte ends and the next begins; 1101 is sent as 00001101.

Converting between binary and denary

To convert binary to denary, write the place values above the bits, multiply each bit by its place value and add the results; only the columns containing 1 contribute. For example, 10011101 is 128 + 16 + 8 + 4 + 1 = 157. There are two reliable methods for converting denary to binary. In the subtraction method you write out the place values, place a 1 under the largest one that is not greater than the number, subtract it and repeat with the remainder until nothing is left, then fill the other columns with 0. In the division method you divide by 2 repeatedly, noting each remainder, until the quotient reaches 0; reading the remainders from the last to the first gives the binary number. Both methods give the same answer, and either earns the method mark if the working is shown. Always check your answer by converting back, and make sure the number of digits matches what the question asks for: '8-bit binary' means exactly eight digits, so add leading zeros where needed. A useful quick check is that every odd number ends in 1 and every even number ends in 0.

Hexadecimal

Hexadecimal is base 16, so it needs sixteen symbols: 0 to 9, then A, B, C, D, E and F for ten to fifteen. Because 16 is 2^4, one hex digit corresponds exactly to one nibble, and one byte is always two hex digits. To convert binary to hex, split the bits into groups of four starting from the right and replace each group with its hex digit; to convert back, expand each digit into four bits. To convert hex to denary, multiply the left digit by 16 and add the right digit, and for denary to hex divide by 16, using the quotient as the first digit and the remainder as the second. Hex is used because it is much shorter than binary, so people can read, write and remember values quickly and make fewer transcription errors, and it converts to binary far more simply than denary does. Typical uses are colour codes in HTML and CSS, MAC addresses, memory addresses shown in debugging tools, error codes and dumps of machine code. Hexadecimal is only a human-friendly notation: inside the computer the same values are still stored and processed as binary, so hex does not save any memory.

Binary-coded decimal

Binary-coded decimal stores each denary digit separately as its own 4-bit code, so 2026 is stored as 0010 0000 0010 0110 rather than as the pure binary value of the whole number. Only the ten patterns 0000 to 1001 are valid; the six patterns 1010 to 1111 are never used. The main advantage is that each nibble can be sent straight to a display digit, which is why BCD is found in digital clocks, calculators, seven-segment displays, meters and timers. It also allows decimal quantities, especially money, to be held exactly, avoiding the rounding errors that occur when values such as 0.1 are converted into binary fractions. The drawbacks are that BCD wastes storage, because six of the sixteen nibble patterns are unused (99 needs 8 bits in BCD but only 7 in pure binary), and arithmetic is more complex, because a result that produces an invalid nibble has to be corrected before it is a valid BCD digit again. A good answer compares BCD with pure binary on these points rather than simply describing it.

T Key terms from the mark schemes

Bit
A binary digit; the smallest unit of data, which stores either 0 or 1.
`Credited: binary digit`

Byte
A group of 8 bits, which can represent 256 different values.

Nibble
A group of 4 bits; half a byte, represented by one hexadecimal digit.

Kilobyte (KB)
1024 (2^{10}) bytes.
`Credited: approximately one thousand bytes`

Megabyte (MB)
1024 kilobytes, or 2^{20} (1 048 576) bytes.

Gigabyte (GB)
1024 megabytes, or 2^{30} bytes.

Terabyte (TB)
1024 gigabytes, or 2^{40} bytes.

Place value
The value of a column in a number, determined by its position and the base, e.g. 1, 2, 4, 8 in binary.
Credited: column weighting

Most significant bit (MSB)
The leftmost bit of a binary number, which has the highest place value.

Least significant bit (LSB)
The rightmost bit of a binary number, with a place value of 1.

Hexadecimal
A base-16 number system using the digits 0–9 and A–F, where each digit represents one nibble (4 bits).
Credited: base 16

Binary-coded decimal (BCD)
A representation in which each denary digit is stored separately as its own 4-bit binary code.

C C# example

Denary to binary: division method

```
int n = 214;
string bits = "";
while (n > 0)
{
    bits = (n % 2) + bits;   // remainder becomes the next bit on the left
    n = n / 2;               // integer division
}
bits = bits.PadLeft(8, '0');  // add leading zeros to make a full byte
Console.WriteLine(bits);      // 11010110
Console.WriteLine(Convert.ToInt32(bits, 2));  // back to 214
```

Exam focus

Conversion questions usually give one mark for method and one for the answer, so always show the place values, the subtractions, the remainders or the nibble split before the final value. Give binary answers in the number of bits asked for, adding leading zeros. For 'how many bits are needed', show the power of 2 you compared against, and remember that 0 counts as a value. When defining units, 'a kilobyte is 1024 bytes' or '2^{10} bytes' is credited; quoting 1000 bytes on its own may not be. 'Explain why hexadecimal is used' typically carries two marks per point, so give a reason and expand it, for example 'it is shorter than binary, so technicians make fewer errors when copying values'. Never claim that hex uses less memory or that the computer stores data in hex – both are marked wrong. For BCD, examiners want the idea that each denary digit is coded separately in four bits, plus a use such as a digital clock. Common mistakes are reading division remainders in the wrong order, forgetting that A = 10 rather than 11, and splitting binary into nibbles from the left instead of the right.

? Check yourself

1. State how many different values can be represented using 6 bits. [1]
2. A sports club in Omagh stores membership numbers from 0 to 1000 as unsigned integers. State the minimum number of bits needed for each membership number and justify your answer. [2]
3. Convert the 8-bit binary number 10011101 to denary. Show your working. [2]

Answers: 1. $2^6 = 64$ (1) 2. 10 bits (1) / 0 to 1000 is 1001 values; $2^9 = 512$ is not enough but $2^{10} = 1024$ is (1) 3. Place values / weightings shown: 128 + 16 + 8 + 4 + 1 (1) / = 157 (1)